URL SHORTENER

System Design Explained with Diagrams and Real-World Examples

A practical design guide

From requirements and short-code generation to redirects, caching, database consistency, analytics, and Snowflake IDs.

Example

A user pastes a long link such as `https://example.com/articles/system-design/how-to-build-a-url-shortener`. The service returns a short link such as `https://sho.rt/aZ81kQ`. When someone opens the short link, the service finds the saved destination and redirects the browser to it.

Includes architecture diagrams • request flows • database schema • design trade-offs • revision questions

1. Requirements analysis

System design starts by deciding what the product must do and what quality targets it must meet. Requirements help determine the architecture; technologies should not be selected before the problem is understood.

Functional requirements — what it must do

- **Create a short URL:** accept a valid long URL and return a short link.
- **Redirect:** when a visitor opens the short link, find the destination and redirect them.
- **Stable mapping:** the same normalized long URL should return the same short URL if that is a product requirement.
- **Optional features:** user-chosen aliases, expiration times, link editing, click analytics, abuse detection, and deletion.

Non-functional requirements — how well it must work

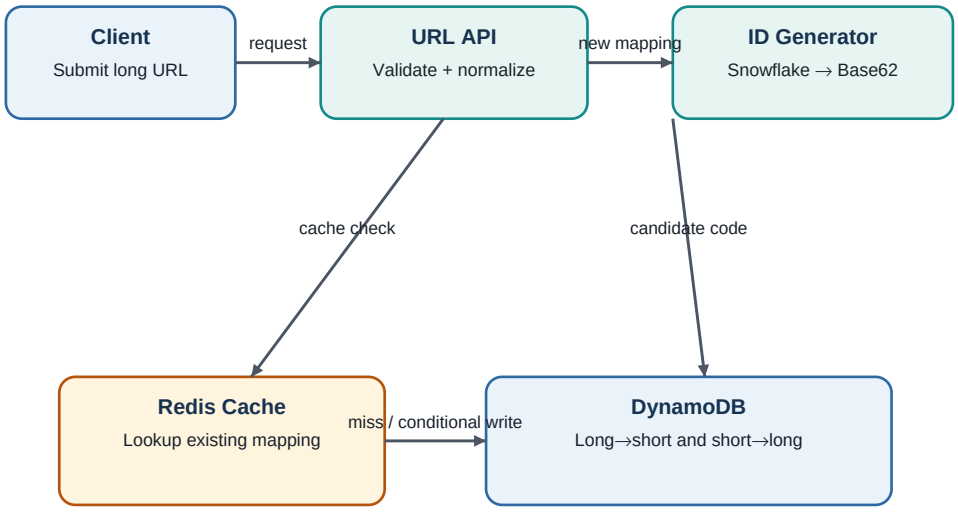
Requirement	Meaning in practice	Design implications
Low latency	Redirects should feel instant; a target might be under 200 ms for the service path.	Cache hot mappings, keep the lookup path short, measure p50/p95/p99 latency.
Uniqueness	Two different links must not accidentally receive the same active short code.	Use a robust ID generator and conditional writes/constraints.
Availability	Users should still be able to redirect during individual instance failures.	Multiple instances, health checks, redundancy, monitoring.
Scalability	Support growth in link creation and especially redirect traffic.	Horizontal scaling, partitioned storage, caching, load balancing.
Durability	Created mappings should not disappear unexpectedly.	Durable storage, backups, recovery and retention policies.

A key workload insight

URL shorteners commonly receive far more redirect requests than link-creation requests. For example, one link might be created once but opened thousands or millions of times. This makes the redirect path a priority for latency, cache efficiency, and availability.

2. High-level architecture

The system can be understood as two main paths: **create a short URL** and **resolve a short URL**. A cache speeds up repeated lookups, a database provides durable mappings, and a message stream handles analytics outside the critical redirect path.



Simplified create flow: normalize first, reuse an existing mapping when present, and atomically create a unique mapping.

Main components

Component	Responsibility
Client	Submits a long URL or opens a short link.
URL API / application service	Validates input, normalizes where safe, checks existing mappings, and orchestrates storage.
Snowflake-style ID generator	Creates unique numeric identifiers across machines when configured correctly.
Base62 encoder	Converts the numeric ID into a compact URL-friendly string.
Redis cache	Stores frequently used mappings for fast lookup.
DynamoDB or another durable database	Persists mappings and supports the access patterns needed by the product.
Kafka and analytics worker	Process click events asynchronously, without making the browser wait for analytics.

3. URL normalization and short-code generation

Normalization aims to avoid treating equivalent input strings as unrelated links. But URL normalization must be conservative: changing a URL incorrectly can send users to a different destination.

What is usually safe?

- Trim leading and trailing whitespace pasted around the URL.
- Parse the URL and validate its scheme, host, and structure.
- Normalize the hostname case, because DNS hostnames are case-insensitive.
- Remove a default port when it is truly equivalent, if your URL library handles it safely.

What should not be changed casually?

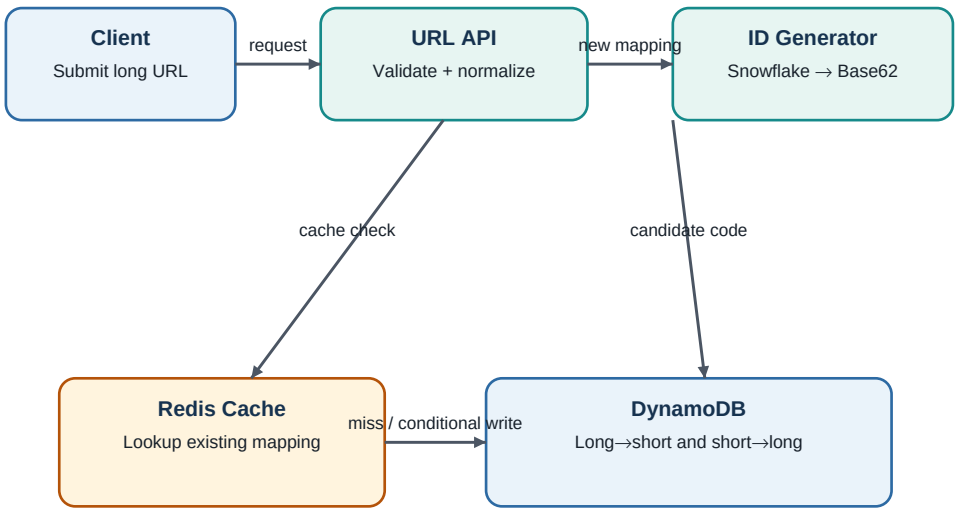
- Path and query values can be case-sensitive on some servers.
- Query parameter order, repeated parameters, percent-encoding, trailing slashes, and fragments can affect application behavior.
- Do not remove “extra symbols” without understanding their meaning. A safer first version may only trim whitespace and use a well-tested URL parser.

Example

```
Input: HTTPS://Example.COM:443/docs?id=7
A conservative normalizer may trim surrounding spaces and canonicalize the
scheme/hostname/default HTTPS port. It should preserve meaningful path and query details rather
than assume all URLs with similar-looking text are identical.
```

Create-short-URL flow

1. Client sends the long URL to the create endpoint.
2. API validates and normalizes the URL.
3. Service checks Redis or a durable long-URL lookup for an existing mapping.
4. If a mapping exists, return its short code.
5. Otherwise, generate a candidate Snowflake ID and encode it with Base62.
6. Attempt an atomic/conditional database write so competing requests cannot create conflicting mappings.
7. If another request won the race for the same long URL, read and return the winner's mapping.
8. Cache the mapping and return the short URL.



Simplified create flow: normalize first, reuse an existing mapping when present, and atomically create a unique mapping.

Important consistency detail

A cache check alone cannot guarantee that the same long URL always maps to one short URL: two requests can both miss the cache at the same time. The durable store needs a conditional/atomic operation or uniqueness mechanism keyed by the normalized long URL. If the write loses a race, the service should return the already-created mapping rather than a second code.

4. Base62 encoding and short links

Base62 uses 62 characters—typically `0–9`, `a–z`, and `A–Z`—to represent an integer compactly. It is a number representation, not encryption.

For example, a numeric ID can be repeatedly divided by 62; the remainders map to Base62 characters. A larger integer generally needs fewer characters in Base62 than in decimal. The exact string depends on the alphabet order chosen by the implementation.

Uniqueness comes from the ID generator, not Base62 itself. If two machines encode the same numeric ID, they produce the same code. Also, sequential IDs can be guessable; if enumeration is a concern, consider a random or permuted code strategy, abuse controls, and access policies.

5. Snowflake-style ID generation

The transcript describes a 64-bit Snowflake-style layout: a sign bit, timestamp bits, machine/worker bits, and sequence bits. The common layout given here is 1 + 41 + 10 + 12 = 64 bits.





Base62 encodes the integer; it does not itself guarantee uniqueness—the ID generator must do that.

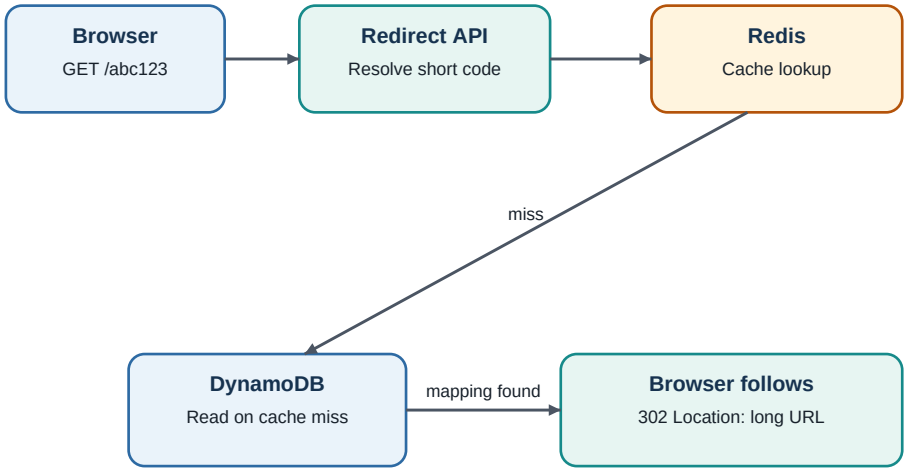
Field	Bits	Purpose
Sign	1	Kept at 0 so the ID is non-negative in the intended signed representation.
Timestamp	41	Time since a chosen custom epoch, usually measured in milliseconds.
Machine / worker ID	10	Separates generators; 10 bits allow 1,024 distinct worker values if allocated correctly.
Sequence	12	Distinguishes IDs generated by one worker in the same millisecond; 12 bits allow 4,096 values per millisecond in the theoretical layout.

Operational cautions

- Worker IDs must be assigned without collisions. Two active generators with the same worker ID can generate duplicate IDs under the same timestamp and sequence conditions.
- Clock rollback can cause duplicate or out-of-order IDs unless the generator detects and handles it.
- The exact timestamp epoch and bit allocation are implementation-specific; the 41-bit timestamp does not mean all implementations share the same start date.
- The sequence limit is per worker per millisecond. At capacity, the generator must wait for the next millisecond or use another defined overflow strategy.

6. Redirection flow and HTTP status codes

When a browser opens a short link, the redirect service resolves the short code to a destination URL and sends an HTTP redirect response.



A 302 response tells the browser to request the destination; cache hits avoid a database read.

Step by step

1. Browser requests `GET /aZ81kQ`.
2. Redirect service extracts the code and checks Redis.
3. On a cache hit, it obtains the destination immediately.
4. On a cache miss, it reads the short-code mapping from the database and may populate the cache.
5. It checks expiration and applicable safety rules.
6. It returns a redirect response with a `Location` header containing the destination URL.
7. The browser makes a new request to the destination.

302 vs. 301

Status	Meaning	Typical consideration for a URL shortener
301 Moved Permanently	The resource has permanently moved.	Browsers and intermediaries may cache the redirect, which can reduce future visits to the shortener and affect analytics or link-change behavior.
302 Found	Temporary redirect.	Often chosen when the service wants each visit to reach the backend so it can apply current rules, track clicks, or honor changes/expiry.

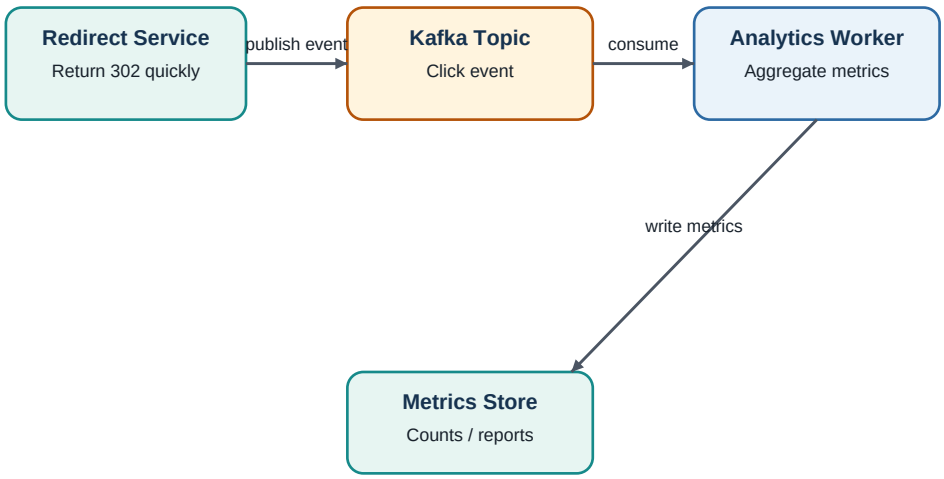
302 is not the only valid option. 307 and 308 preserve the request method more explicitly, while 301/302 have historical method-changing behavior for some clients. For ordinary short-link GET requests, 302 is a common choice. Cache-control headers and client behavior still matter; a 302 does not guarantee that every request will always reach the backend.

Expiration behavior

Store an expiry time if links can expire. When the code is expired, return a deliberate result—such as a 404 or 410 page—rather than redirecting. Decide whether expiry is checked on every request, through cache TTLs, or both; a cache entry must not outlive the permitted link lifetime.

7. Analytics and abuse prevention

Click analytics should usually not make a user wait for a dashboard counter to update. The redirect service can publish a click event and return the redirect while a separate worker aggregates metrics.



Analytics processing is decoupled from the redirect response path.

Example click event fields

A practical event might include the short code, timestamp, referrer (if available), coarse device/browser category, and a privacy-conscious geographic estimate if justified. Avoid collecting more personal data than needed, and define retention and access rules.

Abuse prevention can include rate limiting, malicious-domain detection, spam reports, phishing checks, bot filtering, account controls, and a way to disable dangerous links. These checks should be designed so the redirect path remains reliable while high-risk links can be blocked.

Why Kafka or a queue?

- Decouples click tracking from the immediate redirect response.
- Buffers bursts so analytics workers can catch up later.
- Allows independent scaling of consumers and separate pipelines for reporting or fraud detection.
- Requires retry policies, duplicate-event handling, monitoring of consumer lag, and a plan for events that repeatedly fail.

A queue does not make analytics automatically exactly-once. Events may be delivered more than once depending on the design, so consumers should tolerate duplicates or use deduplication where accurate counts matter.

8. Database schema and access patterns

The service needs to look up a destination from a short code and, if stable mapping is required, look up an existing short code from a normalized long URL. These are two different access patterns.

Table / item collection	Partition or primary key	Example attributes	Purpose
URLByShort	ShortCode	LongURL, CreatedAt, ExpiresAt, OwnerId, Status	Fast redirect lookup by short code.
URLByLong	NormalizedLongURLKey	ShortCode, CreatedAt, ExpiresAt	Reuse the existing short code for the same canonical URL.

In a NoSQL database such as DynamoDB, the actual schema should be designed around access patterns, key size constraints, throughput, partition distribution, and query behavior. Very long URLs may be unsuitable as raw primary keys, so a stable hash of the normalized URL can be used as the lookup key, while the full URL is stored as an attribute. Hash collisions must be handled safely by verifying the original normalized URL.

Conditional writes and race conditions

Suppose Alice and Bob submit the same long URL at nearly the same time. Both may observe that no mapping exists. Each generates a candidate short code. The database must atomically allow only one mapping for the long-URL key. The losing request reads the winning item and returns that same short code. Separately, the short-code key must also be protected against accidental collision.

This is a race condition: the result depends on overlapping operations. A cache does not prevent it because two requests can both miss. Use conditional puts, uniqueness constraints, transactions, or an equivalent durable concurrency control mechanism supported by the chosen store.

9. Reliability, scaling and performance

Challenge	Possible response	Caveat
Hot short links receive many clicks	Cache popular mappings; use CDN/edge caching only if redirect semantics permit.	Redirect analytics, expiration, and link updates can be affected by edge caching.
Database read load rises	Cache-aside reads, well-distributed keys, suitable indexes and capacity planning.	Cache misses and cache outages can create a thundering herd.
Traffic spikes	Horizontally scale stateless API instances and use a load balancer.	Scaling has a delay; protect dependencies and cap costs.
Redis becomes unavailable	Fall back to the database with timeouts and controlled load shedding.	A sudden full cache miss can overload the database.
Kafka consumers fall behind	Scale consumers, monitor lag, and tune batches/partitions.	More consumers do not help beyond the available partition parallelism.
Malformed or malicious links	Validate URLs, scan risky destinations, rate-limit creation, and support takedowns.	Validation should not accidentally reject legitimate URL forms.

Latency targets should be measured, not assumed. Track percentiles such as p50, p95, and p99 for redirect responses, cache hit rate, database latency, error rates, queue lag, and availability. A stated goal of under 200 ms is a target to validate in the actual environment, not a guarantee produced by the architecture alone.

10. Security and product considerations

- **Open redirect abuse:** a URL shortener intentionally redirects to external destinations, so it can be abused for phishing. Add reputation checks, abuse reporting, throttling, and takedown procedures.
- **SSRF protection:** if the service fetches or previews destination URLs, restrict requests to internal/private IP ranges and guard against DNS rebinding. A service that only stores and redirects URLs has a different risk profile from one that fetches them.
- **Privacy:** define what click data is collected, who can see it, and how long it is retained.
- **Custom aliases:** reserve aliases atomically and decide how to handle collisions, profanity, impersonation, and reserved paths.

- **Expiration and deletion:** decide whether expired mappings remain for analytics or are deleted, and make sure caches respect the policy.

11. End-to-end worked example

A small company wants to shorten this URL: `https://shop.example/products/blue-shoes?campaign=summer`.

Creation

- The API trims surrounding whitespace and validates the URL.
- It computes the product's chosen normalized lookup key.
- The service checks whether that long URL already has a short code.
- If not, the Snowflake-style generator creates a unique numeric ID.
- Base62 converts that ID to a compact code, for example `aZ81kQ` (illustrative only).
- Conditional database writes persist both lookup directions without allowing duplicate mappings.
- The service caches the mapping and returns `https://sho.rt/aZ81kQ`.

Redirection

- A visitor opens `https://sho.rt/aZ81kQ`.
- Redis returns the long URL on a cache hit, or the service reads DynamoDB on a miss.
- The redirect service checks the expiry and status of the link.
- It publishes a click event to Kafka when available and returns HTTP 302 with a `Location` header.
- An analytics worker consumes the event and updates click metrics independently.

What if two users create it simultaneously?

Both requests can generate different candidate IDs, but only one conditional write should win for the normalized long-URL key. The losing request reads the winning mapping and returns the same short URL. The database must also prevent a short-code collision.

12. Key trade-offs to remember

Decision	Benefit	Cost / risk
Cache redirects	Lower latency and fewer database reads.	Stale mappings, invalidation work, cache-outage load spikes.
302 redirects	Allows backend logic and analytics on visits more often than a permanently cached redirect.	Extra request to the shortener; clients and intermediaries may still cache according to headers.
Snowflake IDs + Base62	Fast distributed numeric IDs and compact codes.	Worker coordination, clock issues, and potentially guessable codes.
Two lookup tables / indexes	Supports both long→short and short→long access patterns.	Extra write coordination and storage.
Async analytics	Keeps analytics off the critical response path.	Delayed metrics, duplicates, queue operations.
DynamoDB / NoSQL	Can scale key-based lookups horizontally.	Data modeling and conditional-write behavior must match access patterns; cost and limits need planning.

13. Revision questions

- Why does a URL shortener usually need both long-to-short and short-to-long lookups?
- Why can Redis alone not guarantee that duplicate short URLs are never created?
- What does Base62 do, and what does it not do?
- Why must Snowflake worker IDs be unique among active generators?
- What is the difference between a cache hit and a cache miss in the redirect flow?
- Why might a service choose 302 rather than 301?
- Why should click analytics be processed asynchronously?
- What can happen if cached mappings outlive their expiration time?
- Why must URL normalization be conservative?

Suggested answers

1. One supports idempotent creation/reuse; the other resolves visits. 2. Concurrent requests can both miss the cache; the durable database needs atomic conditional writes or uniqueness controls. 3. Base62 compactly represents an integer; it does not create uniqueness or encryption. 4. Duplicate worker IDs can cause numeric ID collisions. 5. A hit finds the mapping in Redis; a miss reads the durable store. 6. A 302 can help preserve backend control over redirects and analytics; behavior depends on client and cache rules. 7. It avoids making the browser wait for metric aggregation. 8. An expired link could keep redirecting incorrectly. 9. Path/query details may be meaningful and changing them can change the destination.

Final takeaway

A scalable URL shortener is not just an ID encoder. It is a carefully coordinated system: validate inputs, define safe URL equivalence, generate unique codes, enforce atomic mappings, serve redirects quickly, keep caches consistent with expiry and updates, process analytics asynchronously, and protect users from abuse.

This guide expands the transcript provided in the conversation. Diagrams and examples are illustrative teaching aids, not a claim about any particular company's production implementation.